

Basic C++ & Java I/O Programming

1. What program libraries do you need to include in a C++ or a Java ?

(a) in C++: `#include <iostream> // when use console I/O.`
 `#include <fstream> // when use file I/O.`
 `#include <string> // when use string functions.`
 `#include <cmath> // when use math functions.`
 `using namespace std; //So you do not need to use std:: in your code.`

(b) in Java: `import java.io.*; // when use file I/O.`
 `import java.util.*; // when use console I/O and other build-in functions.`

2. Declare an input file variable, `inFile`, to open the file from 1st argument of the commend line.

(a) in C++: `// public int main (int argc; char** argv)`
 `ifstream inFile;`
 `inFile.open (argv [1]);`

or

`ifstream inFile (argv [1]);`

(b) in Java: `// public static void main (String [] args)`

`Scanner inFile = new Scanner (new FileReader (args [0]));`
 `// to read one integer - inFile.nextInt();`
 `// to read one word - inFile.next ();`
 `// to read one text-line - inFile.nextLine();`

or `FileReader inFile = new FileReader (args [0])`
 `// read one char - inFile.read(), return unicode`
 `// return -1 at the end of file.`

or `BufferedReader inFile = new BufferedReader (new FileReader (args [0]));`
 `// read one char - inFile.read (); return Unicode; return -1 at the end of file.`
 `// read one textline – inFile.readLine(); return the whole text line as a string;`
 `// return -1 at the end of file.`

3. Declare an output file variable, `outFile`, to open the file from 2nd argument of the commend line.

(a) in C++: `// public int main (int argc; char** argv)`
 `ofstream outFile;`
 `outFile.open (argv [2]);`

or

`ofstream outFile (argv [2]);`

(b) in Java: `// public static void main (String [] args)`

`PrintWriter outFile1 = new PrintWriter (args [1])`
 `// outFile1.print (data) // write data; data can be integer or string.`
 `// outFile1.println (data) // write data plus newline`

or `FileWriter outFile1 = new FileWriter (args [1])`
 `// outFile1.write (data) if data is a string data type.`
 `// outFile1.write (integer.toString(data)) // if data is an integer`

or `BufferedWriter outFile1 = new BufferedWriter (new FileWriter (args [1]));`

```
// outFile1.write (data) if data is string data type.
// outFile1.write (integer.toString (data)) // if data is an integer
```

4. How to check the arguments in the command line correct and check to see if a file can be opened (text file)

(a) in C++:

```
#include <fstream>
using namespace std;

int main (int argc, char** argv){
    if (argc != 3) { // check the command line's parameters (executable + 2 parameters)
        cout << "command line needs 2 parameters";
        exit (1);
    }
    ifstream inFile (argv [1]) // Open the input file from the first string of the command line
    if (!inFile) // or if (!inFile.is_open()) {
        cout << "input file cannot open";
        exit (1);
    }
    ofstream outFile (argv [2]) // Open the output file from the second string of the command line
    if (!outFile) // or if (!outFile.is_open()) {
        cout << "output file cannot open";
        exit (1);
    }
}
```

(b) in Java:

```
import java.io.*; // when use file I/O.
import java.util.*; // when use console I/O and other build-in functions.
```

```
class Main {
    public static void main (String args []) throws IOException{
        if (args.length != 2) {
            System.out.println ("Program needs 2 arguments");
            System.exit(1);
        }

        Scanner inFile = null;
        try {
            inFile = new Scanner (new FileReader (args [0]));
            System.out.println ("inFile opened successfully");

        } catch (FileNotFoundException e) {
            System.out.println ("inFile cannot be opened");
            System.exit(1);
        }

        BufferedWriter outFile = null;
        try { outFile = new BufferedWriter (new FileWriter (args [1]));
            System.out.println ("outFile opened successfully");

        } catch (IOException e) {
            System.out.println ("outFile cannot be opened");
            System.exit(1)
        }
    }
}
```

5. Suppose inFile is defined as below, how to read an integer, intData, from inFile?

- (a) in C++: `ifstream inFile (argv [1]);`
 `int intData;`
 `inFile >> intData;`
- (b) in Java: `Scanner inFile = new Scanner (new FileReader (args [0]));`
 `int intData = inFile.nextInt();`

6. Suppose `inFile` is defined as below, how to read a character, `chrData`, from `inFile`?

- (a) in C++: `ifstream inFile (argv [1]);`
 `char chrData;`

 `inFile >> chrData; // skip spaces ' ', tab '\t', and newlines '\n'.`
or
 `ifstream inFile (argv [1]);`
 `char chrData = inFile.get (chrData); // does not skip spaces ' ', tab '\t', and newlines '\n'.`
- (b) in Java: `FileReader inFile = new FileReader (args [0]);`
 `char chrData = (char) inFile.read ();`
or
 `BufferedReader inFile = new BufferedReader (new FileReader (args [0]));`
 `char chrData = (char) inFile.read ();`

7. Suppose `inFile` is defined as below, how to read a string, `strData`, from `inFile`?

- (a) in C++: `ifstream inFile (argv [1]);`
 `string strData;`
 `inFile >> strData;`
- (b) in Java: `Scanner inFile = new Scanner (new FileReader (args [0]));`
 `string strData = inFile.next();`

8. Suppose `outFile` is defined as below, write a character, `chrData`, to `outFile`?

- (a) in C++: `ofstream outFile (argv [2]);`
 `char chrData;`
 `outFile << chrData;`

or
 `ofstream outFile (argv [2]);`
 `char chrData = inFile.get(chrData);`
- (b) in Java: `PrintWriter outFile = new PrintWriter (new FileWriter (args [1]));`
 `outFile.print (chrData);`
or
 `FileWriter outFile = new FileWriter (args [1]);`
 `char chrData;`
 `outFile.write (chrData);`

or
 `BufferedWriter outFile = new BufferedWriter (new FileWriter (args [1]));`
 `char chrData;`
 `outFile.write (chrData);`

9. How to read data to the end of a file?

- (a) in C++:

```

datatype data; // datatype may be int or char or string
while (inFile >> data){ // assume the inFile has opened.
}

```

(b) in Java:

```

int intData;
while (inFile.hasNextInt()){
    intData = inFile.nextInt();
}
string strData;
while (inFile.hasNext()){
    strData = inFile.next();
}

```

10. To close a file (let the name of file be inFile)

(a) in C++: inFile.close();

(b) in Java: inFile.close();

11. How to declare an integer, intData, and read it from console?

(a) in C++: int intData;
cin >> intData;

(b) in Java: Scanner inCon = new Scanner (System.in);
int intData = inCon.nextInt ();

12. Suppose intData is defined as below, how to write intData, to console?

(a) in C++: int intData = 10;
cout << intData;

(b) in Java: int intData = 10;
System.out.println (intData);

13. Suppose strData is defined as below, how to read strData, from console? // read one word.

(a) in C++: string strData;
cin >> strData;

(b) in Java: Scanner inCon = new Scanner (System.in)
String strData = inCon.next ();

14. Suppose strData is defined as below, how to write strData, to console?

(a) in C++: string strData = "Hello World";
cout << strData;

(b) in Java: String strData = "Hello World";
System.out.println(strData);

15. How to declare a character, chrData, and read it from console?

(a) in C++: char chrData;
cin >> chrData;

(b) in Java: char chrData;
 Scanner inCon = new Scanner (System.in);
 inCon.useDelimiter (“”) // this makes the scanner scan char by char instead of string by string
 chrData = inCon.next();

16. How to declare a character, chrData, and write the character to console

(a) in C++: char chrData = ‘t’;
 cout << chrData;

(b) in Java: char chrData = ‘t’;
 System.out.println(chrData);

17. Statically declare a one-dimensional (1-D) integer array of size 100.

(a) in C++: int intAry [100];

(b) in Java: int[] intAry = new int[100];

18. Statically declare a 1-D character array of size 100

(a) in C++: char chrAry [100];

(b) in Java: char [] chrAry = new char[100];

19. **Dynamically** declare a 1-D **integer array** of size N (let the array be named – intData)

(a) in C++: int* intAry = new int[N];

(b) in Java: int [] intAry = new int[N];

20. **Dynamically** declare a 1-D **character array** of size N (known).

(a) in C++: char* chrAry = new char[N];

(b) in Java: char [] chrAry = new char[N];

21. **Dynamically** declare a 2-D **integer array** of size Rows by Cols. (let the array be named – intAry).

(a) in C++: int** int2DAry = new int* [Rows];
 for (int i = 0; i< Rows; i++)
 int2DAry[i] = new int [Cols];

(b) in Java: int[][] int2DAry = new int [Rows][Cols];

22. **Dynamically** declare a 2-D **character array** of size Rows by Cols. (let the array be named – charAry).

(a) in C++: char** chr2DAry = new char*[Rows];
 for (int i = 0; i< Rows; i++)
 chr2DAry[i] = new char [Cols];

(b) in Java: int[][] chr2DAry[i] = new char [Rows][Cols];

23. How to convert an integer to string:

(a) in C++: int intData = 123;
 string stringData = to_string (intData);

(b) in Java: `int intData = 123;`
 `String stringData = String.valueOf (intData);`

24. How to convert a string to integer:

(a) in C++: `string stringData = "123";`
 `int intData = stoi (stringData);`

(b) in Java: `string stringData = "123";`
 `int intData = Integer.parseInt (stringData);`

25. How to convert a character to integer:

(a) in C++: `char chrData = '3';`
 `int intData = atoi (chrData);`

 or `int intData = (int) chrData`

(b) in Java: `char chrData = '3';`
 `int intData = (int) chrData;`

26. How to compare two strings:

(a) in C++: `string str1;`
 `string str2;`
 `cin >> str1; or inFile >> str1;`
 `cin >>str2; or inFile >> str2;`

`if str1 < str2 // return true, if str1 < str2, otherwise, return false.`
 `if str1 > str2 // return true, if str1 > str2, otherwise, return false.`
 `if str1 == str2 // return true, if str1 == str2, otherwise, return false.`

(b) in Java: `string str1;`
 `string str2;`
 `Scanner inCon = new Scanner (System.in) // console input`
 `String str1 = inCon.next ();`
 `String str2 = inCon.next ();`

`if (str1.equals (str2) // returns true if they are the same, otherwise, return false.`
 `if (str1.compareTo (str2)) // returns 0 if str1 is the same as str2`
 `// returns negative if str1 comes before str2`
 `// returns positive if str1 comes after str2`